

Technical Project Report

1. Project overview

PixelGrid is an Arduino-based LED matrix games project for a custom PixelGrid board. The repository follows an Arduino Sketchbook structure: runnable sketches are stored in `Games`, hardware support sketches are stored in `pixeltest` and `joysticks`, local libraries are stored in `libraries`, and setup/tutorial documentation is stored in `Tutorial`.

The main application evidence is concentrated in two games:

- `Games/Tetris`: a Tetris implementation with input handling, rendering, host-runtime serial support, optional score submission, and host-side tests.
- `Games/Breakout`: a Breakout implementation with modular game, input, rendering, and pin configuration files.

The project uses a custom local library, `libraries/PixelGridcore`, to abstract LED matrix cells, seven-segment-style LCD digits, panels, buttons, and shapes. The project also vendors third-party Arduino libraries such as Adafruit NeoPixel, FastLED, and Firmata.

2. Problem being solved

The project solves the problem of turning an LED matrix and attached controls into a reusable embedded games platform. Instead of writing every sketch directly against individual LED indexes, the repository provides:

- A grid abstraction for mapping logical rows and columns onto a serpentine LED strip layout.
- Game-specific logic for Tetris and Breakout.
- Input abstractions for buttons and joystick movement.
- Rendering abstractions for LED matrix output and a six-digit segment display.
- Tutorial and setup documentation for board configuration and upload workflows.
- Host-side unit tests for selected Tetris game logic without requiring the physical device.

3. Major features

3.1 Tetris game

`Games/Tetris` implements a full embedded Tetris game loop. Evidence includes:

- Tetromino masks and rotations in `Games/Tetris/Game.h`.
- Board state, current piece, next piece, hold piece, score, level, and fall timing in `TetrisGame`.
- Collision validation, line clearing, scoring, leveling, soft drop, ghost-piece calculation, and piece locking in `Games/Tetris/Game.h`.
- Button/joystick debouncing and repeat movement in `Games/Tetris/Input.h`.
- LED matrix and LCD panel rendering in `Games/Tetris/Render.h`.
- Runtime mode management in `Games/Tetris/Tetris.ino`, including standalone mode and host mode.
- Serial host frame parsing in `Games/Tetris/HostRuntime.cpp`.

- Optional Wi-Fi score submission in `Games/Tetris/NetSubmit.h`.
- Host-side tests in `tests/tetris_game_tests.cpp`.

3.2 Breakout game

`Games/Breakout` implements a second game.

- Game state and rules in `Games/Breakout/Game.cpp` and `Games/Breakout/Game.h`.
- Input scanning and repeat behaviour in `Games/Breakout/Input.cpp` and `Games/Breakout/Input.h`.
- Rendering routines in `Games/Breakout/Render.cpp` and `Games/Breakout/Render.h`.
- Hardware constants in `Games/Breakout/Pins.h`.
- Arduino setup and frame-loop orchestration in `Games/Breakout/Breakout.ino`.

3.3 Reusable PixelGrid library

`libraries/PixelGridcore` provides reusable display components:

- `Pixel_Grid`: maps logical grid coordinates to physical LED strip indexes and buffers colours.
- `LCD_Digit`: renders digits and segment masks onto a seven-segment-style LED display.
- `LCD_Panel`: manages multiple `LCD_Digit` objects.
- `Shape`: represents reusable grid shapes.
- `Button`: supports shared button behaviour.

3.4 Host-based testing and CI

The repository includes a host test strategy for Tetris. `tests/tetris_game_tests.cpp` compiles game logic with stub Arduino and renderer headers from `tests/stubs`. `.github/workflows/tetris-tests.yml` builds and runs the tests on GitHub Actions for changes affecting Tetris and tests.

4. Technical challenges

4.1 Embedded rendering constraints

LED matrix games require deterministic updates, careful mapping between logical game coordinates and physical LED strip indexes, and efficient frame rendering. The `Pixel_Grid` conversion table handles serpentine mapping so game code can work with row/column coordinates.

4.2 Input debouncing and repeat behaviour

Mechanical buttons and joystick directions can produce noisy signals. The input modules implement debounced stable states, edge detection, and joystick repeat timing so game controls feel responsive without repeated false triggers.

4.3 Game-state correctness

Tetris requires accurate collision detection, piece rotation, line clearing, score calculation, level progression, hold-piece rules, soft drop timing, and lock behaviour. Breakout requires paddle movement, ball physics, brick collision, speed changes, brick drops, scoring, and game-over handling.

4.4 Hardware-independent tests

Embedded projects are often hard to test without hardware. This repository addresses that by using host stubs in `tests/stubs` so selected Tetris rules can be tested with `g++`.

4.5 Optional network score submission

`Games/Tetris/NetSubmit.h` signs score submissions with HMAC-SHA256, uses NTP-derived timestamps, configures enterprise Wi-Fi, and posts JSON to an external API endpoint. This introduces security, timing, network reliability, and configuration challenges.

4.6 Host runtime integration

`Games/Tetris/HostRuntime.cpp` supports serial packets for external host-driven rendering. This requires packet framing, length checks, resynchronisation after invalid data, and timeout-based fallback to standalone mode.

5. Implementation approach

The implementation uses a layered embedded architecture:

1. Hardware constants define pins, dimensions, and timing values.
2. Input modules convert raw pin reads into debounced game actions.
3. Game modules maintain in-memory state and implement rules.
4. Render modules convert game state into PixelGridCore display calls.
5. The Arduino `.ino` entry points initialise hardware, instantiate components, and run the main loop.
6. Optional integration layers support host serial frames and HTTP score submission.

For Tetris, much of the gameplay is encapsulated in the `TetrisGame` struct in `Games/Tetris/Game.h`. For Breakout, gameplay is separated into global state and functions across `Game.cpp` and `Game.h`.

6. Technical achievement

The project demonstrates technical achievement in several areas:

- Embedded C++ development using Arduino-compatible APIs.
- Real-time game-loop design for constrained hardware.
- LED matrix abstraction over a custom physical layout.
- Reusable library design for PixelGrid components.
- Multiple game implementations using common hardware concepts.
- Debounced input handling and joystick repeat behaviour.
- Host-mode serial protocol support for external rendering control.
- Optional secure score-submission flow using Wi-Fi, TLS client classes, NTP, HMAC-SHA256, base64url encoding, timestamp validation assumptions, and nonce generation.
- Host-side unit tests and CI automation for selected gameplay rules.
- Existing setup and tutorial documentation for board use.

7. Limitation(s)

- Some vendored libraries are included directly under `libraries`, increasing repository size and making dependency version governance dependent on manual updates.

8. Future improvements

- Add `NetConfig.example.h` documenting required configuration values without committing secrets.
- Add OpenAPI documentation for the external score service
- Add PlatformIO or Arduino CLI configuration to pin board packages and library versions.
- Expand host-side tests to cover Breakout logic and more Tetris edge cases.
- Add a serial protocol specification and test fixtures for host-mode packets.
- Replace `setInsecure()` with certificate validation or documented development/production configuration modes.
- Add diagrams for the physical wiring and LED index mapping in editable source form in addition to the existing PDF.